

更快的 NEMU

TL;DR

本文记录对 NEMU（一个面向教学场景的解释型指令集模拟器）的一次优化实践。本文首先通过 **CPU Isolation** 构建实验环境，确定 NEMU 的性能测量方法，然后针对本次实验过程开发了实验日志管理工具 **explog**。最后本文以 microbench 的 ref 规模作为 workload，通过**指令缓存**、**基本块缓存**、**direct threading** 等手段对 NEMU 进行优化后，最终取得了 **25.26x** 的加速比。相关代码和实验数据已公开¹。

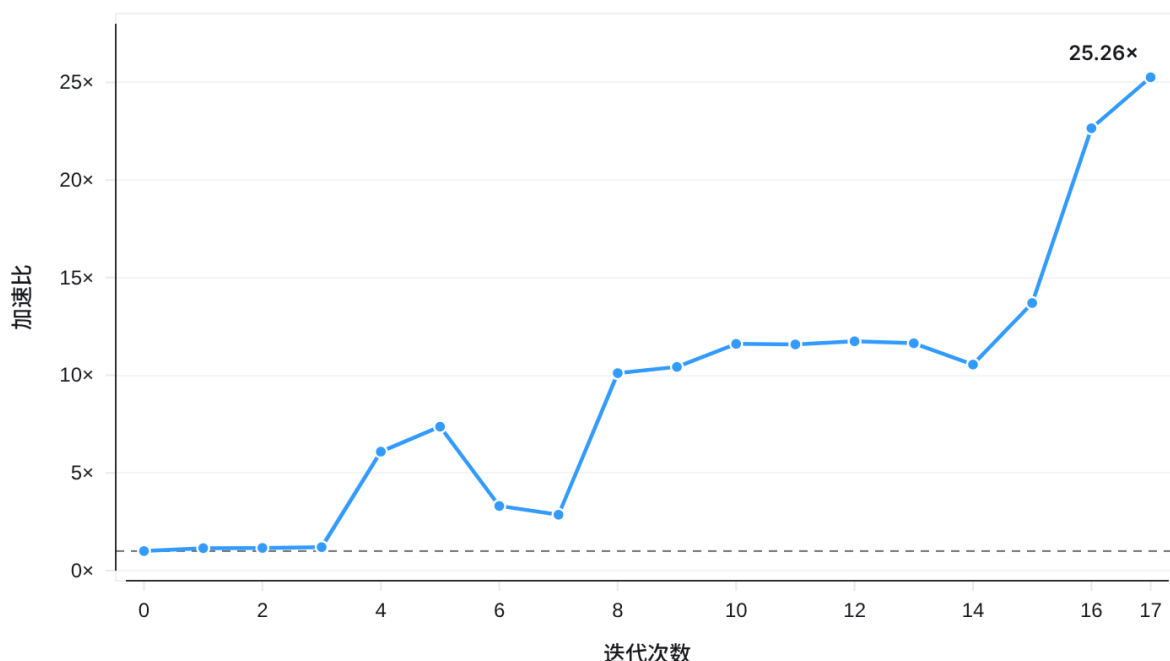


图 1 迭代过程概览

类别	baseline	优化后	变化
Host Instructions	5.55×10^{11}	2.67×10^{10}	-95.20%
Host Cycles	2.25×10^{11}	8.96×10^9	-96.01%
Branch	1.56×10^{11}	5.04×10^9	-96.76%
Branch Miss	9.17×10^7	7.73×10^6	-91.57%
仿真频率	3.34×10^7 inst/s	8.46×10^8 inst/s	+2434.63%

表 1 性能变化概览

¹代码仓库: <https://github.com/GlowingCurve/nemu-dev>

目录

介绍	1
动机和目标	1
背景	1
限定条件	2
测量 NEMU 的性能	2
实验环境	3
统计方法	3
性能测量方法	4
实验基础设施 —— explog	4
核心模型	4
存储结构	5
执行过程	5
NEMU 的优化过程	6
原始 NEMU 的性能	6
编译器优化	6
移除设备轮询	7
译码瓶颈	8
检查译码实现	8
guest PC 局部性	8
重构执行路径	9
InstCache	10
首版 InstCache	10
命中率与缓存容量	10
减少数据搬运	11
更好的 Cache	11
适配 BasicBlock	11
BasicBlockCache	12
首版实现与优化	12
瓶颈分析	13
Direct threading	14
块间连续分派	14
总结和讨论	15
总结	15
讨论	15
附录	16

介绍

动机和目标

一个没有经过任何优化的 NEMU，其运行 ref 规模的 microbench 用时在 50 到 60 秒左右。

虽然这个性能在 NEMU 所面向的场景中已经够用，但是 NEMU 运行 microbench 究竟可以运行多快？虽然这个问题很难说有多大的实际意义，但这出于纯粹的好奇心。

因此，本文的目标为：**以 microbench 作为 workload 的情况下，尝试优化 NEMU，尽可能提高 NEMU 的运行速度。**

背景

NEMU 是一款面向教学场景的解释型指令集模拟器，主要用于“一生一芯”项目和南京大学计算机系统基础实验中，支持多种指令集和设备的搭配。

microbench 是“一生一芯”及南京大学计算机系统基础实验提供的 Benchmark 之一。

测试规模	指令数	使用场景
test	约 300K	正确性测试
train	约 60M	在 RTL 仿真环境中研究微结构行为
ref	约 2B	在模拟器或 FPGA 环境中评估处理器性能
huge	约 50B	衡量高性能处理器(如真机)的性能

表 2 microbench 提供的测试规模

名称	描述
qsort	快速排序随机整数数组
queen	位运算实现的 n 皇后问题
bf	Brainf**k 解释器，快速排序输入的字符串
fib	Fibonacci 数列 $f(n)=f(n-1)+\dots+f(n-m)$ 的矩阵求解
sieve	Eratosthenes 筛法求素数
15pz	A*算法求解 4x4 数码问题
dinic	Dinic 算法求解二分图最大流
lzip	Lzip 数据压缩
ssort	Skew 算法后缀排序
md5	计算长随机字符串的 MD5 校验和

表 3 microbench 所包含的子项

如表 2 表 3 所述，microbench 规模适中，ref 规模下原始 NEMU 的运行时间可控；不依赖操作系统：覆盖面广，测试程序质量高，是用来评估 NEMU 性能的理想 Benchmark。

限定条件

本文所选择的 NEMU 来自我实际的一生一芯项目仓库。作为 baseline 的原始 NEMU 面向日常使用和开发场景，没有经过任何的优化，可以运行 RT-Thread，是我进行一生一芯 C 阶段答辩时的配置。

本文做出如下限定：

- NEMU 采用 RV32IM 指令集，设备只考虑串口和时钟，其余设备不考虑。
- 只针对 microbench 一个 workload 进行优化，暂不考虑其他工作负载。
- 优化过程中不考虑 gdb, trace, diff test, watchpoint 等工具的兼容性，NEMU 始终运行在批处理模式下。

实验平台信息如下表所述：

类别	配置
操作系统	Fedora Linux 44 Workstation
Linux 内核版本	7.1.8-200.fc44.x86_64
处理器	AMD Ryzen 9 9955HX @ 5.46GHz
内存	32 GiB DDR5-5600
Host 编译器	GCC 16.1.1 20260515, Red Hat 16.1.1-2
交叉编译器	riscv64-linux-gnu-gcc 16.1.0

表 4 实验平台与编译器配置

测量 NEMU 的性能

想让 NEMU 跑得更快，首先必须知道 NEMU 究竟跑得有多快。

直接运行 NEMU 时，它只是 Linux 上的一个普通进程。调度器可能将它迁移到不同的逻辑 CPU，硬件中断和其他系统任务可能打断它，CPU 频率也会随系统状态发生变化。这些因素与 NEMU 自身的实现无关，却会影响测得的运行时间。本文将这类外部扰动统称为**系统噪声**。

系统噪声主要带来两个问题。首先，同一版本的多次测量结果会产生额外波动，需要更多样本才能得到稳定的性能估计。其次，当两个版本的性能十分接近时，观察到的差异可能来自运行环境，而不是代码改动本身。因此，在进行优化之前，需要先建立一个稳定、可重复的实验环境。

实验环境

Linux 提供了一组 CPU Isolation 机制，可以减少目标 CPU 上与实验无关的系统活动。本文选择了一套临时的隔离方案，采用运行时修改、重启即恢复的策略。

针对前述噪声来源，实验环境进行了三方面控制：

1. **调度噪声**: 通过 cgroup v2 的 cpuset partition 将一个完整物理核心划入 isolated partition, 使其与普通工作负载的调度域分离。与此同时，调整 per-CPU 内核线程和 watchdog 的 CPU 掩码，尽量避免内核延迟任务和 watchdog 事务在目标 CPU 上执行。
2. **频率噪声**: 通过 cpufreq 接口将目标 CPU 的调频器和能效偏好设置为最高性能的 performance 挡，同时将最高频率限制为 4 GHz。实验平台 CPU 最高加速频率可达 5.46 GHz，将最高频率限制在 4 GHz 有助于长时间保持 CPU 频率，可以减少动态调频以及功耗、温度变化对测量结果的影响。
3. **中断噪声**: 重新配置中断亲和性，将可以迁移的中断尽量移出隔离 CPU，减少实验运行期间由外部设备和系统活动造成的打断。

这套方案不能消除全部系统噪声，因为部分中断无法迁移，CPU 的实际运行频率也仍会发生小幅波动。因此，剩余的不确定性依然需要通过重复采样和统计指标进行描述。

统计方法

本文使用 microbench 输出的程序执行时间 (Scored Time) 作为 NEMU 的性能指标。Scored Time 记录各子项的运行时间之和，不包含 NEMU 启动、字符打印等外围过程。Scored Time 越短，说明 NEMU 执行该 workload 的速度越快。

样本与性能指标

NEMU 运行一次得到一个 Scored Time 样本，记为 x_i 。对于同一版本，本文重复运行 n 次 microbench，并使用样本均值 $\bar{x}$ 描述该版本的平均运行时间。

版本 B 相对版本 A 的加速比定义为：

$$S_{A \rightarrow B} = \frac{\bar{x}_A}{\bar{x}_B}$$

后文若无特殊说明，“运行时间”均指 Scored Time 的样本均值。加速比使用相应版本的样本均值计算。

测量结果描述

为了描述样本均值的不确定性，本文基于 Student's t 分布计算总体平均运行时间的双侧 99% 置信区间 (Confidence Interval, CI)。置信区间同时受到样本波动和样本数量的影响：样本波动越小、样本数量越多，得到的区间通常越窄。

本文进一步使用相对误差界 (Relative Margin of Error, RMOE) 描述置信区间相对于样本均值的宽度。99% RMOE 定义为 99% 置信区间的半宽度与样本均值的比值。相比直接比较置信区间宽度，RMOE 可以更直观地比较不同实验的测量不确定性。

性能测量方法

对于每个待测版本，本文最多运行 125 次 microbench。前 5 次运行作为预热样本丢弃，不参与统计；此后至少采集 10 个有效样本，即每个版本至少运行 15 次。

为避免在结果已经足够稳定时继续投入测量时间，本文认为当 99%RMOE 小于 1% 时，当前结果就已经足够稳定，可以停止继续采样。因此本文从第 10 个有效样本开始，每得到一个样本，重新计算样本均值、双侧 99% CI 和 99% RMOE。当 99% RMOE 小于 1% 时，则停止采样；若始终未达到该条件，则在取得 120 个有效样本后停止。

由于采样过程会反复检查 RMOE，并根据当前结果决定是否停止，最终计算的 Student's t 区间不保证仍具有固定样本条件下严格的 99% 覆盖率。

后文若无特殊说明，所有性能数据均在相同的隔离环境下，按照上述采样方法获得；若无明显标识，则表明在取得 120 个有效样本前 99%RMOE 就已经小于 1%。

实验基础设施 —— explog

在性能实验中，代码改动的结果需要实验进行评估，实验又产生对应的原始数据和处理结果，后续实验又可能从此前的某个版本继续展开。如果缺乏合适的工具，随着实验数量增加，实验数据的维护和管理将越来越困难。

Git 虽然可以记录代码变化，但无法直接维护代码、实验、原始数据、处理结果之间的关系。如果只保留最终采用的改动，失败实验及其结果将难以追溯；如果手动保存每一次实验的全部状态，实验过程本身又会产生较高的管理成本，拖累实验进行。

针对这一问题，本文实现了实验日志管理工具 explog。

核心模型

explog 将一次完整实验表示为一个 ExperimentNode。一个实验节点既描述一次实验时的代码状态，也描述实验数据。

在 `explog` 的模型中，每个实验节点最多指向一个父节点，表示当前实验从哪个已有实验继续修改。如果一个实验节点没有父节点，那么该实验节点就是根节点。从同一个节点尝试不同优化方向时，会产生多个子节点。由此，整个优化过程被组织为一棵实验树。

在实验树下，基于父子关系可以还原一个实验基于哪些改动产生；通过比较同一父节点下的多个子节点，也可以直接判断不同方案相对于同一基线的性能变化。最终采用的优化路径和被放弃的实验路径在实验树下都可以被自然地保留，便于进行实验过程的复盘。

相应的，为了确定一个实验节点，`explog` 需要记录三类信息：

1. 节点自身的信息，包括实验 ID、完成时间和实验说明；
2. 节点的父节点 ID；
3. 节点对应的代码状态和实验数据。

存储结构

`explog` 在 Git 仓库根目录下运行，使用配置文件、实验日志和实验数据目录保存状态。

配置文件 `explog.toml` 定义实验数据根目录、实验日志目录，数据采集命令和数据处理命令。实验日志采用 JSONL 格式保存，每一行对应一个 `ExperimentNode`。

字段	描述
<code>id</code>	实验节点的唯一标识
<code>parent_id</code>	父节点 ID，根节点为 <code>null</code>
<code>timestamp</code>	实验完成时间
<code>message</code>	实验说明
<code>git_commit</code>	实验运行前 HEAD 指向的 commit
<code>git_diff_path</code>	相对于仓库根目录的完整工作区 diff
<code>data_dir</code>	该节点对应的实验数据目录

表 5 `ExperimentNode` 的字段描述

其中，`git_commit` 只能确定已提交的代码状态，无法描述实验时尚未提交的修改。因此，`explog` 同时保存 HEAD commit 和完整的工作区 diff，两者共同描述实验实际使用的代码。

JSONL 日志只保存节点元数据和文件路径。体积较大的代码 diff、原始数据和处理结果仍保存在普通文件中。

执行过程

执行一次实验时，`explog` 依次完成以下步骤：

1. 加载并校验配置文件，检查实验 ID、父节点 ID、目标数据目录和实验日志路径；
2. 执行 git 命令，使未被忽略的 untracked 文件可以进入 diff；

3. 执行命令，使用 git 捕获当前工作区相对于 HEAD 的完整修改；
4. 创建本次实验的数据目录，并将代码修改保存为 git.diff；
5. 运行数据采集命令
6. 运行数据处理命令
7. 构造对应的 ExperimentNode，并将其追加到 JSONL 实验日志。

实验节点只在数据采集和处理命令全部成功后写入正式日志。如果其中任意命令失败或异常，explog 不会写入实验节点，但已有输出仍会保留，用于定位失败原因。

因此，JSONL 中的每个节点都对应一次完整执行的实验；失败过程则保留现场。

NEMU 的优化过程

原始 NEMU 的性能

为了在后续实验中同时采集 host instructions、cycles 等硬件性能计数器，首先需要确认 perf 是否会对 Scored Time 产生可辨认的影响。

场景	n	均值/s	99% CI/s	RMOE/%
无 perf	10	47.9889	47.9711–48.0068	0.0372
有 perf	10	47.9815	47.9527–48.0102	0.0599

表 6 原始 NEMU 的 Scored Time

不使用 perf 时，10 个有效样本的平均 Scored Time 为 47.9889 s；使用 perf 时，平均 Scored Time 为 47.9815 s。两组均值相差约 0.0156%，极为微小，所以本文认为该场景下 perf 开销可以忽略。后续实验统一使用 perf 采集 host instructions、cycles、branches 和 branch misses，并使用 Scored Time 比较不同版本的运行性能。

后文所称原始 baseline 均为 perf 包裹下测量得到的 baseline。

编译器优化

原始 baseline 使用 -Og 编译。本文首先测试编译器优化能够提供多少性能收益。

优化	加速比	用时	Host Instructions	Host Cycle
baseline	1.00×	47.98 s	5.55×10^{11}	2.25×10^{11}
-O2	1.15×	41.62 s	3.59×10^{11}	1.94×10^{11}
-O2 + LTO	1.16×	41.22 s	3.34×10^{11}	1.92×10^{11}
-O3 + LTO	1.20×	40.06 s	3.11×10^{11}	1.87×10^{11}

表 7 不同编译器优化相对于 baseline 的性能变化

以上结果说明编译器消除了相当一部分 host 侧指令。但从 -O2 继续启用 LTO(Link-time Optimization)、再提升至 -O3，带来的增量收益已经明显减小。此时继续调整编译选项很难取得明显的性能变化，需要进一步分析 NEMU 的性能。

移除设备轮询

在 -O3 + LTO 版本上进行 perf 采样后，发现 85.35% 的 cycles 落在 get_time 调用链中：

# Overhead	Command	Shared Object	Symbol
#			
#			
85.35%	riscv32-nemu-in	[vdso]	[.] 0x0000000000000f09
	---0x7f20125f7f09		
	get_time_internal (inlined)		
	get_time (inlined)		
	device_update (inlined)		
	execute (inlined)		
	cpu_exec.part.0.constprop.0		
	cpu_exec (inlined)		
	cmd_c (inlined)		
	sdb_mainloop (inlined)		
	engine_start (inlined)		
	main		
	__libc_start_call_main		
	__libc_start_main@@GLIBC_2.34		
	_start		
4.71%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] cpu_exec.part.0.constprop.0
...			

代码 1 perf 观察到的热点代码及调用链局部

NEMU 启用设备支持后，原始执行循环每执行一条 guest instruction，都会调用一次 device_update()。该函数首先读取时间，判断是否需要更新已经注册的设备；满足更新条件时，再更新键盘、VGA 等设备。

本文使用的 microbench 不注册也不访问键盘和 VGA，程序获取时间是通过 MMIO 读取当前时间完成的，因此不需要在执行过程中更新设备。但在原始执行路径中，每条 guest instruction 仍会调用 device_update()，并通过 get_time() 检查更新时间。对于当前 workload，这部分工作不会改变程序执行结果，却在执行过程中被大量重复。

因此，本文在 microbench 使用的 NEMU 配置中，将 device_update() 移除。做出该改动后，NEMU 不再适用于需要设备更新的运行场景。

优化配置	加速比	用时	Host Instructions	Host Cycle
-O3 + LTO	1.20×	40.06 s	3.11×10^{11}	1.87×10^{11}
移除 device_update	6.09×	7.88 s	1.84×10^{11}	3.62×10^{10}

表 8 移除 device_update 前后的性能变化

结果表明移除 `device_update` 实现了较大的性能提升。

译码瓶颈

移除 `device_update()` 后，原有热点已经消失。重新进行 `perf` 采样。

# Overhead	Command	Shared Object	Symbol
#			
#			
42.00%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] decode_exec.isra.0
41.95%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] cpu_exec.part.0.constprop.0
6.48%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] decode_operand.constprop.1.isra.0
5.86%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] decode_operand.constprop.0.isra.0
2.51%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] paddr_read
1.09%	riscv32-nemu-in	riscv32-nemu-interpreter	[.] paddr_write
...			
#			

代码 2 移除 `device_update` 的程序热点局部

可以观察到 `decode_exec`、`cpu_exec` 和 `decode_operand` 成为主要热点，其中 `decode_exec` 和 `cpu_exec` 分别占采样 `cycles` 的 42.00% 和 41.95%。性能瓶颈转移到 NEMU 的指令执行路径。

检查译码实现

为了判断译码函数中是否还存在明显的实现开销，本文进一步检查了与译码有关的汇编。

...				
6.75 :	22c8:	andl	\$0x7f, %edx	
1.60 :	22cb:	cmpl	\$0x37, %edx	
0.00 :	22ce:	je	0x2550 <decode_exec.isra.0+0x2a0>	
0.00 :	22d4:	cmpl	\$0x17, %edx	
0.36 :	22d7:	je	0x2518 <decode_exec.isra.0+0x268>	
0.00 :	22dd:	cmpl	\$0x6f, %edx	
0.29 :	22e0:	je	0x25e0 <decode_exec.isra.0+0x330>	
0.00 :	22e6:	movl	%r9d, %edx	
2.00 :	22e9:	andl	\$0x707f, %edx	
0.00 :	22ef:	cmpl	\$0x67, %edx	
0.00 :	22f2:	je	0x2640 <decode_exec.isra.0+0x390>	
...				

代码 3 `decode_exec.isra.0` 汇编代码片段

NEMU 源码中的模式匹配逻辑已经被编译器转换为一组比较和条件跳转。继续优化 `decoder` 只能尝试降低一次译码的开销，性能提升空间极为有限。

因此，本文尝试减少译码过程发生的次数。

guest PC 局部性

考察 `microbench` 执行过程中每个 `guest PC` 的出现次数，并按照出现频率从高到低排序。

类别	数据
出现五次以上的 PC 个数	2964
出现第 1024 多的 PC 出现次数	35932
前 1024 多的 PC 出现次数之和	1.84×10^{11}
前 1024 多的 PC 出现次数占总动态指令数的比例	99.4%

表 9 guest PC 统计指标

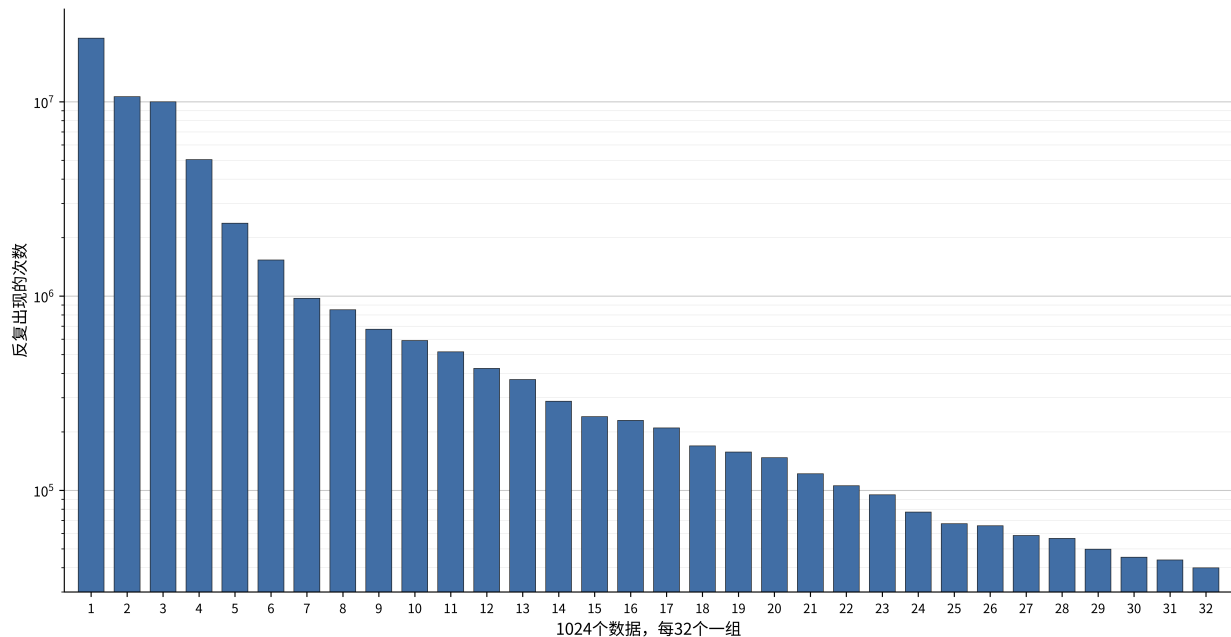


图 2 由高到低 guest PC 出现次数统计（对数坐标）

上述结果说明，microbench 的执行集中在一个很小的 PC 集合中。由于 microbench 不包括自修改代码，这一结果说明 NEMU 正在对少量相同指令进行大量重复译码。

既然绝大部分动态指令都来自已经执行过的 PC，与其继续降低每次译码的成本，不如缓存得到的译码结果，在后续执行中复用。这就是指令缓存(InstCache)。

重构执行路径

原始 NEMU 将译码和执行组织在同一条路径中。为了保存并复用译码结果，需要先将指令的译码信息表示为独立对象，并将指令语义拆分为可以被缓存调用的执行函数。

本文分三个阶段完成这一重构。第一阶段将指令语义拆分为独立执行函数，同时简化外层执行循环和指令计数逻辑。该版本的 Scored Time 从 7.875 s 降至 6.511 s，相对原始 baseline 的总加速比达到 7.37x。

后续两个阶段引入 Inst 对象，用于保存寄存器索引、立即数和执行函数指针。由于 InstCache 尚未启用，但执行路径需要进行对象构造、数据搬运和间接调用，Scored Time 分别退化到 14.506 s 和 16.778 s。

优化	加速比	用时	Host Instructions	Host Cycle
移除 device_update	6.09×	7.88 s	1.84×10^{11}	3.62×10^{10}
重构第一阶段	7.37×	6.51 s	1.53×10^{11}	2.99×10^{10}
重构第二阶段	3.31×	14.51 s	2.02×10^{11}	6.75×10^{10}
重构第三阶段	2.86×	16.78 s	2.14×10^{11}	7.81×10^{10}

表 10 三个重构阶段的性能变化

InstCache

首版 InstCache

第一版 InstCache 包含 1024 个缓存项，采用直接映射结构。缓存使用 PC 的低位进行索引，并使用完整 PC 作为 tag。每个缓存项保存译码结果：PC、源寄存器和目标寄存器索引、立即数以及执行函数指针 handler。

执行一条 guest instruction 时，首先根据当前 PC 查找缓存。tag 匹配时，直接使用缓存项中的译码信息调用 handler；未命中时，则重新译码当前指令并填充对应缓存项。

优化	加速比	用时	Host Instructions	Host Cycle
重构第三阶段	2.86×	16.78 s	2.14×10^{11}	7.81×10^{10}
InstCache	10.11×	4.74 s	9.89×10^{10}	2.23×10^{10}

表 11 引入 InstCache 前后的性能变化

即使与重构前移除 device_update() 的 7.875 s 相比，InstCache 仍然带来了约 1.66x 的增量加速。这说明执行路径重构引入的成本远低于重复译码的开销。

命中率与缓存容量

运行记录显示，1024 项 InstCache 的命中率约为 99.916%。为了判断缓存容量是否是当前性能的瓶颈，本文进一步测试了不同容量：

缓存容量	加速比	用时	Host Instructions	Host Cycle
512 项	10.05×	4.78 s	9.95×10^{10}	2.24×10^{10}
1024 项	10.11×	4.74 s	9.89×10^{10}	2.23×10^{10}
2048 项	10.17×	4.72 s	9.89×10^{10}	2.22×10^{10}

表 12 不同容量 InstCache 的性能变化

三种容量之间的性能差异很小。说明对于当前 workload，1024 项 InstCache 已经能够覆盖主要的动态执行路径，继续扩大容量不会带来明显收益。

减少数据搬运

检查 InstCache 命中路径的汇编后，发现命中时会复制整个 Inst 结构体。由于命中率已经接近 100%，这次复制也几乎会在每条动态 guest instruction 上发生，成为新的重复开销。

本文将命中路径修改为直接持有缓存项指针，并通过指针访问译码信息、调用对应的 handler，避免复制完整的 Inst。修改后，Scored Time 从 4.74 s 降至 4.132 s，总加速比从 10.11x 提升到 11.61x。

优化配置	加速比	用时	Host Instructions	Host Cycle
InstCache	10.11×	4.74 s	9.89×10^{10}	2.23×10^{10}
Inst 优化后	11.61×	4.13 s	6.57×10^{10}	1.94×10^{10}

表 13 InstCache 优化后的性能变化

进一步观察发现，NEMU 此时会启动 Runtime Check, 会使用 assert 检查程序行为。由于本文围绕 microbench 开展优化，程序行为已经确认合法，Runtime Check 还会带来额外开销，所以本文关闭 Runtime Check，性能提升较为有限，具体见附录中表 18。

更好的 Cache

优化后的 InstCache 已经消除了绝大部分重复译码，但 perf 显示，热点开始集中到逐指令缓存查找和命中判断。

对于顺序执行的代码，解释器会反复经过相同的指令序列。即使序列中的每条指令都能命中 InstCache，NEMU 仍然需要为每条指令计算索引、读取缓存项并比较 PC tag。既然一段顺序控制流通常会作为整体重复出现，这些逐指令检查就成了重复工作。

因此，可以将缓存和命中判断的单位从单条指令扩大到多条连续指令，即基本块 (**BasicBlock**)。进入 BasicBlock 时只根据起始 PC 进行一次缓存查找；命中后，块内指令按照已经保存的顺序连续执行，不再逐条检查 InstCache。**BasicBlock** 在可能改变顺序控制流的指令处结束。

因此本文构建基本块缓存(**BasicBlockCache**)。

适配 BasicBlock

BasicBlockCache 会在一次命中后连续执行多条指令，原先“每条指令结束时统一处理”的状态更新必须进行修改。

保持 x0

RISC-V 的 x0 寄存器必须始终为 0，对 x0 的写入应当被丢弃。原先的实现允许指令临时写入 x0，并在每条指令执行结束后重新将其置为 0。

这一实现不能直接用于块级执行。如果一个基本块中的前一条指令写入 `x0`，后一条指令又读取 `x0`，只在基本块结束时清零会使后一条指令读到错误的值。

一种直接做法是在每个执行函数中判断目标寄存器是否为 `x0`，但这会让几乎所有寄存器写回路径增加额外判断。本文添加一个非架构可见的 Dummy 寄存器槽 `x32`，在原有 32 个 GPR 之外，译码时将所有以 `x0` 为目标寄存器的写入重定向到 `x32`。`x0` 在初始化时被显式写入 0，此后不再作为任何指令的写入目标。

这样，对 `x0` 的写入会落入 Dummy 槽，后续指令读取 `x0` 时仍然得到 0，同时不需要增加逐指令清零操作。

移除 `mstatus` 写回

原先为了适配 `DiffTest`，NEMU 在每条 `guest instruction` 执行后都会将 `mstatus` 写回 `0x1800`。本文使用的 `microbench` 不包含 CSR 相关指令，在当前执行路径中，`mstatus` 始终保持固定值。

因此，本文将 `mstatus = 0x1800` 移到初始化阶段执行，移除逐指令写回。

这些迭代的性能变化见附录中的优化概览表。

opcode

`InstCache` 使用函数指针 `handler` 表示指令语义。它避免了重复译码，但每条动态 `guest instruction` 仍然需要进行一次间接函数调用和一次函数返回。

为了建立统一的块内执行循环，本文将 `handler` 替换为 `uint8_t opcode`。译码阶段只记录 `opcode` 和操作数信息；执行阶段通过 `switch` 根据 `opcode` 进入对应的指令语义。

优化配置	加速比	用时	Host Instructions	Host Cycle
Inst 优化后	11.61×	4.13 s	6.57×10^{10}	1.94×10^{10}
适配 BasicBlock	11.64×	4.12 s	7.01×10^{10}	1.93×10^{10}

表 14 适配后的性能变化

适配过程没有直接带来明显加速，但已经为 `BasicBlockCache` 提供了实现基础。

BasicBlockCache

首版实现与优化

第一版 `BasicBlockCache` 允许一个基本块保存最多 32 条指令及其译码信息。实现中，整个 `BasicBlock` 结构体被按值传递给执行函数。由于结构体包含一组 `Inst`，一次缓存命中会触发较大规模的数据复制。

首版 `BasicBlockCache` 发生性能退化: `Scored Time` 从 4.121 s 退化到 4.549 s，总加速比从 11.64x 退化至 10.55x；`RMOE` 异常，120 次有效采样后依然为 1.929%。性能退

化的可能因为块级缓存减少了逐指令缓存查找，却被命中路径中的结构体复制抵消。而 RMOE 异常可能与 IPC 下降，分支指令增多对于 CPU 微结构不友好有关。

这一结果与此前 InstCache 的经验一致：缓存项只需要被访问，不应该在命中时被完整复制。将 BasicBlock 的按值传递改为指针传递后，Scored Time 降至 3.501 s，相对适配版本获得约 1.18x 增量加速。

优化配置	加速比	用时	Host Instructions	Host Cycle	Branch	Branch Miss
适配 BasicBlockCache	11.64×	4.12 s	7.01×10^{10}	1.93×10^{10}	1.63×10^{10}	1.78×10^7
首版 basicblockcache	10.55×	4.55 s	7.63×10^{10}	2.12×10^{10}	1.72×10^{10}	1.84×10^7
优化 basicblockCache	13.70×	3.50 s	6.13×10^{10}	1.64×10^{10}	1.40×10^{10}	1.91×10^7

表 15 BasicBlockCache 及其优化相对于 baseline 的性能变化

瓶颈分析

完成指针化后，本文继续测试 BasicBlockCache 的两个结构参数。

将最大基本块长度从 32 条指令降低到 16 条后，Scored Time 为 3.504 s，与原配置基本相同。这说明当前 workload 中长度超过 16 条的热基本块没有贡献显著的额外收益，最大块长度不是当前程序的性能瓶颈。

将缓存容量从 1024 项扩大到 16384 项后，Scored Time 为 3.502 s，同样没有明显变化。这说明当前缓存容量已经能够覆盖主要执行路径，继续扩容不能解决剩余瓶颈，故重新进行 perf 采样。

# Overhead	Command	Shared Object	Symbol
#			
#			
69.38%	riscv32-nemu-in	riscv32-nemu-interpret	[.] switch_execution
26.83%	riscv32-nemu-in	riscv32-nemu-interpret	[.] isa_exec_once
2.19%	riscv32-nemu-in	riscv32-nemu-interpret	[.] paddr_write
1.39%	riscv32-nemu-in	riscv32-nemu-interpret	[.] cpu_exec.part.0.constprop.0

代码 4 perf 报告局部

: 0	0x3040	<switch_execution>:
4.54 :	3040:	pushq %rbp
2.31 :	3041:	movq %rsp, %rbp
0.00 :	3044:	subq \$0x40, %rsp
10.09 :	3048:	cmpl \$0x32, 0xc(%rdi)
5.30 :	304c:	ja 0x39a8 <switch_execution+0x968>
0.00 :	3052:	movzbl 0xc(%rdi), %eax
3.37 :	3056:	leaq 0x53d3(%rip), %rdx # 0x8430
4.32 :	305d:	movslq (%rdx,%rax,4), %rax
5.07 :	3061:	addq %rdx, %rax
3.60 :	3064:	jmpq *%rax
...		

代码 5 switch_execution 的跳转表处部分采样信息

switch_execution 占采样 cycles 的 69.38%，成为当前最主要的热点。生成的汇编表明，编译器将 opcode switch 实现为跳转表。

BasicBlockCache 已经将缓存查找从每条指令一次降低为每个基本块一次，但块内的每条指令仍然需要经过 switch 跳转表和循环控制：读取 opcode、检查取值范围、计算目标地址、间接跳转到执行逻辑，再返回循环处理下一条指令。

此时，新的瓶颈变成了执行已译码指令时的控制分派开销。

Direct threading

本文使用 GCC 的 labels-as-values 扩展实现 **Direct threading**。初始化时建立 opcode 到局部标签地址的映射；完成译码后，每个 Inst 直接保存对应指令语义的标签地址。执行完当前指令后，通过 computed goto 跳转到下一条 Inst 保存的目标标签。

实际的 Direct threading 的执行过程可以概括为：执行当前指令语义；读取下一条指令的目标标签；直接跳转到下一条指令语义。

优化配置	加速比	用时	Host Instructions	Host Cycle	Branch	Branch Miss
优化 basicblockCache	13.70×	3.50 s	6.13×10^{10}	1.64×10^{10}	1.40×10^{10}	1.91×10^7
direct-threading	22.65×	2.12 s	3.37×10^{10}	1.00×10^{10}	6.50×10^9	1.14×10^7

表 16 引入 direct-threading 带来的性能变化

引入 Direct threading 后相对原始 baseline 的总加速比从 13.70x 提升到 22.65x。分支指令和分支预测错误的数量都大幅下降，证明 Direct Threading 大幅优化了控制流，发挥了重要作用。

块间连续分派

Direct threading 削减了基本块内部的控制开销，但一个基本块执行结束后，NEMU 仍然需要返回外层循环，由外层循环计算 next PC、查询 BasicBlockCache，再进入目标基本块。对于 microbench 来说，guest PC 极为密集，基本块之间的控制开销也会十分可观。因此，可以在基本块层面进一步做 Direct Threading。

本文做出如下优化：基本块执行结束后，解释器计算 next PC，并直接检查对应的 BasicBlockCache 缓存项；命中时立即进入目标块的第一条指令，未命中时完成 refill 后再进入目标块。这就形成了块间连续分派。

优化配置	加速比	用时	Host Instructions	Host Cycle
direct-threading	22.65×	2.12 s	3.37×10^{10}	1.00×10^{10}
块间连续分派	25.26×	1.90 s	2.67×10^{10}	8.96×10^9

表 17 引入块间连续分派带来的性能变化

引入块间连续分派后，Scored Time 从 2.119 s 降至 1.899 s，相对 direct threading 版本进一步获得约 1.12x 加速，相对原始 baseline 的总加速比达到 25.26x。

总结和讨论

总结

本文以 RV32IM 配置、仅含串口和时钟的 NEMU 为对象，以 microbench ref 规模为 workload，完成了一次性能优化。

在方法上，本文首先通过 CPU Isolation 控制了调度、频率和中断三类系统噪声，建立了稳定可重复的实验环境；以 Scored Time 为性能指标，采用重复采样加双侧 99% 置信区间的方式描述测量不确定性，并在结果足够稳定时停止采样。

为了维护优化过程中的代码、实验与数据之间的关系，本文实现了实验日志管理工具 explog，将整个实验过程组织为一棵实验树，有效管理整体的实验结果。

本文遵循“测量—改动—验证”的策略进行性能优化：编译器优化实现 1.20x 的加速比；移除与当前 workload 无关的设备轮询提高到 6.09x；基于 guest PC 局部性的统计结果，依次引入 InstCache (10.11x)、减少数据复制 (11.61x)、BasicBlockCache 及其优化 (13.70x)、Direct threading (22.65x) 和块间连续分派，最终 Scored Time 从 47.98 s 降至 1.90 s，相对原始 baseline 取得 25.26x 加速。

过程中出现多次性能退化：执行路径重构的中间版本（3.31x、2.86x）和首版 BasicBlockCache (10.55x) 均出现性能退化。过程也出现了多个最终被放弃的实验节点，如 512 项 Inst 缓存、2048 项 Inst 缓存等，具体见附录中表 18。

讨论

进一步优化：实现块间连续分派后，程序热点重新回到指令语义的执行本身。在现有的解释器方案下，执行函数可用的优化手段已经基本用尽，继续提升需要改变执行方式：将热点区域的 guest 代码直接转译为 host 机器码，即引入 JIT。早期探索表明这一方案较 BasicBlockCache 有数量级的提升空间，但其实现所需的工作量远超本文全部优化的总和，且会显著增加 NEMU 的复杂度。因此本文不将其作为本次目标。

NEMU 正确性：引入 BasicBlockCache 后，sdb、difftest、trace 等基础设施不再兼容，正确性验证手段相应减少。但本文的所有优化均不改变指令语义本身：InstCache 与 BasicBlockCache 缓存对程序透明；x0 的 Dummy 寄存器和 mstatus 写回的移除在当前 workload 下均不改变程序行为。microbench 本身包含对结果的检查，而每个优化版本均通过全部测试，说明 NEMU 在当前 Workload 上可以认为是可靠的。如果要进一步优化，则需要对 NEMU 的正确性进行检查。

适用范围: 本文的结果是针对 microbench 一个特定的 workload 而言的: 25.26x 的加速比建立在 microbench 不使用外部设备、不包含自修改代码、guest PC 高度集中的前提下。对需要键盘,VGA 或包含 CSR 操作的其他场景并不适用。

附录

以下给出了完整的迭代历史。描述中含有“*”表明该实验节点最终被放弃,描述中含有“**”表示该实验节点测量得到的 RMOE 出现异常。

描述	均值/s	加速比
baseline	47.9815	1.00×
O2 优化	41.6187	1.15×
开启 LTO	41.2225	1.16×
开启 O3 优化	40.0581	1.20×
移除设备轮询	7.8752	6.09×
拆分执行函数	6.5109	7.37×
引入函数指针	14.5062	3.31×
译码执行解耦	16.7784	2.86×
InstCache	4.7438	10.11×
512 项 InstCache*	4.7751	10.05×
2048 项 InstCache*	4.7177	10.17×
关闭运行时检查	4.6025	10.43×
尝试优化 InstCache*	6.2326	7.70×
优化 InstCache	4.1323	11.61×
第一次尝试引入 opcode*	4.3198	11.11×
移除每指令写回 mstatus	4.1426	11.58×
第二次引入 opcode	4.0879	11.74×
精简译码和 x0 写入	4.1214	11.64×
BasicBlockCache**	4.5492	10.55×
优化 BasicBlockCache	3.5012	13.70×
最大基本块长度改为 16*	3.5037	13.69×
BasicBlockCache 扩容*	3.5019	13.70×
direct threading	2.1185	22.65×
引入块间连续派发	1.8994	25.26×

表 18 最终版本的迭代历史